

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра програмної інженерії

Звіт
З дисципліни «Аналіз та рефакторинг коду»
З лабораторної роботи №2

Виконала:

ст. гр. ПЗПІ-22-9

Терновський Денис

Перевірив:

Дашенков Д. С.

Харків 2024

2 РОЗРОБКА БАЗИ ДАНИХ ДЛЯ СЕРВЕРНОЇ ЧАСТИНИ ПРОГРАМНОЇ СИСТЕМИ ТА ПРИКЛАДНОГО ПРОГРАМНОГО ІНТЕРФЕЙСУ (API).

Мета:

Розробити базу даних для серверної частини програмної системи та прикладного програмного інтерфейсу.

Хід роботи:

Розглянемо будову програмної системи:

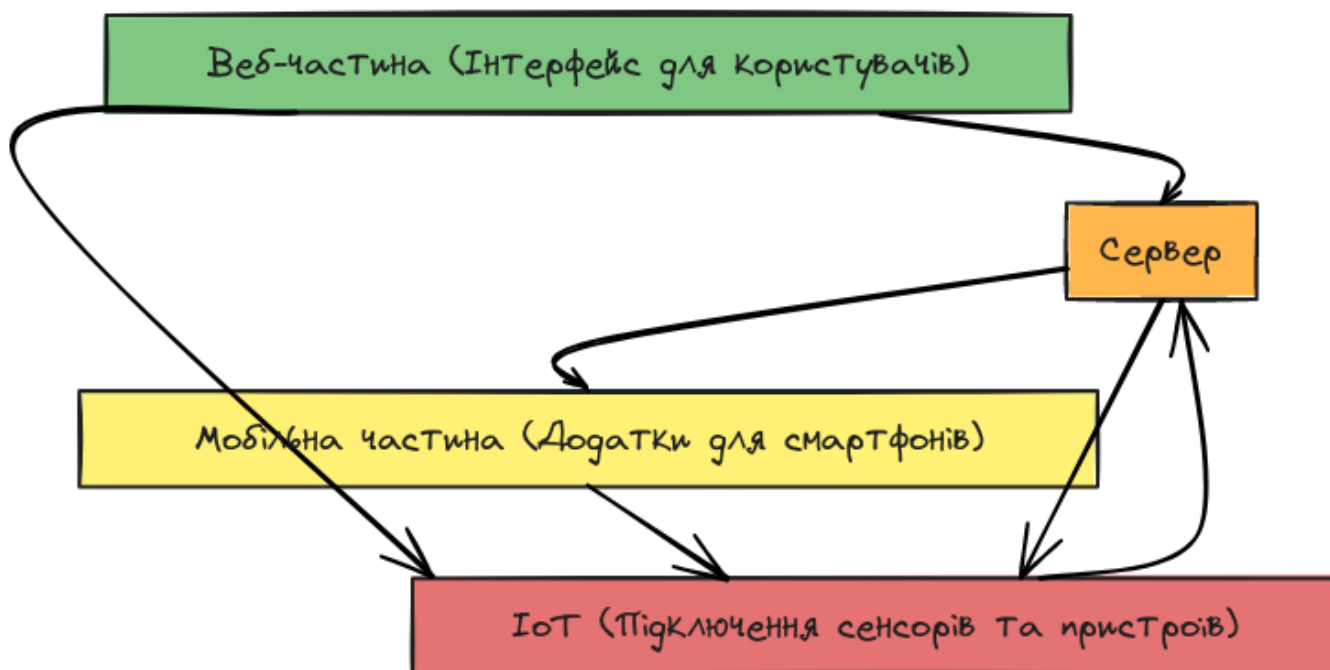


Рисунок 2.1 – Будова програмної системи

Розглянемо Use Case діаграму:

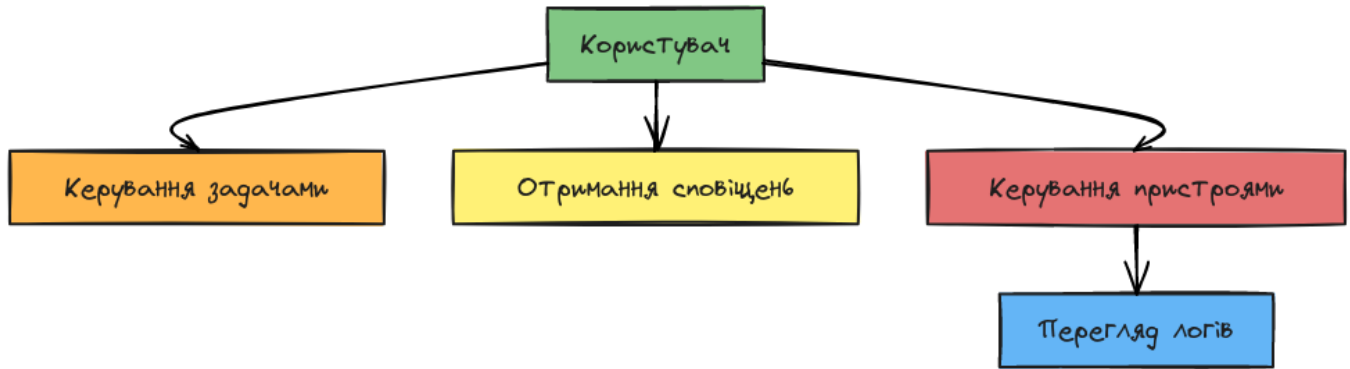


Рисунок 2.2 – Use-case діаграма

Розглянемо:

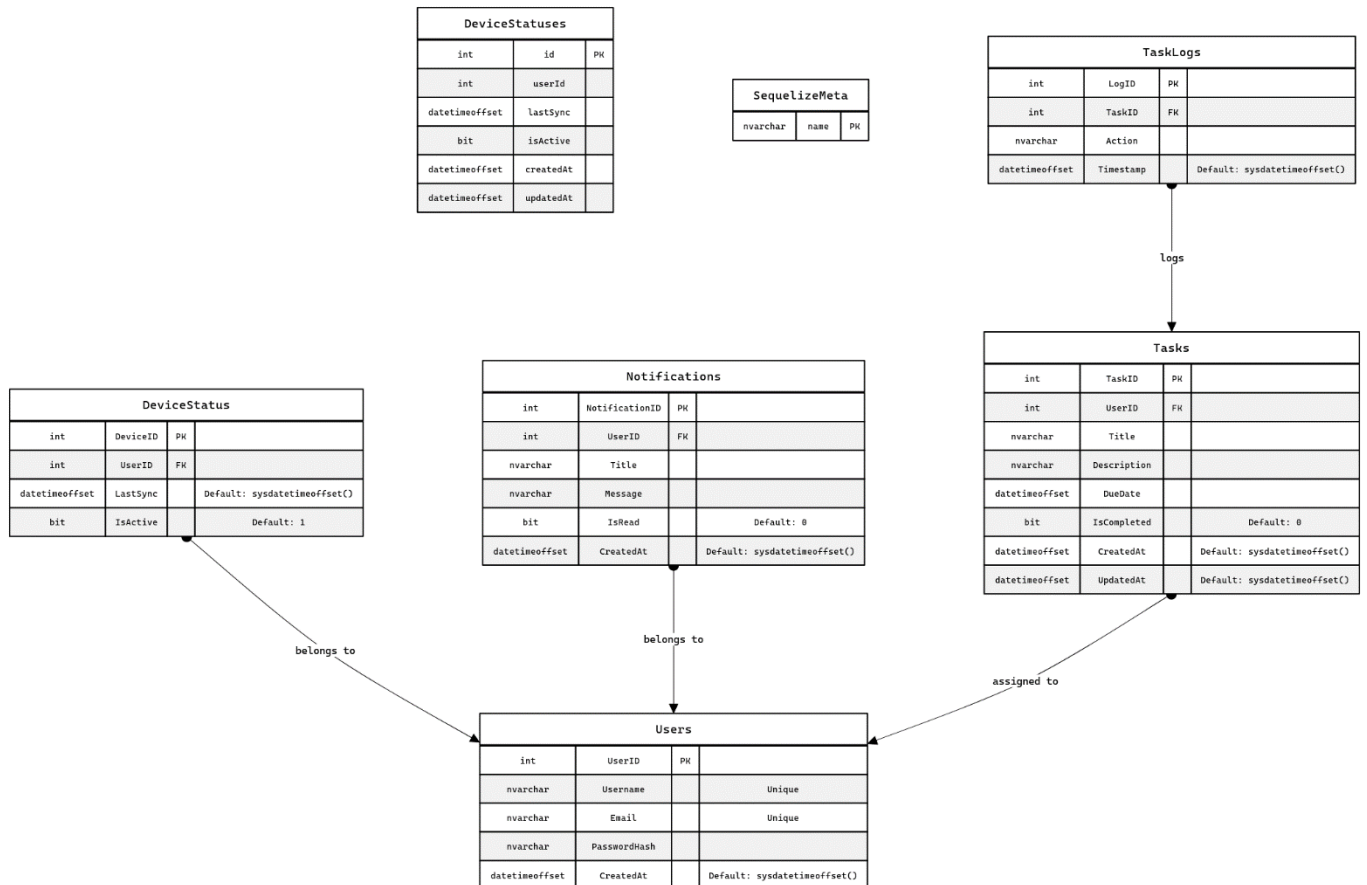


Рисунок 2.3 – Діаграма структури БД

Основні сутності бази даних:

1. Users (Користувачі)

Головна сутність, яка зв'язана:

- З **DeviceStatus** зв'язком один до багатьох (зберігання інформації про пристрої, пов'язані з користувачем).
- З **Notifications** зв'язком один до багатьох (збереження сповіщень для користувача).
- З **Tasks** зв'язком один до багатьох (збереження завдань користувача).

2. **DeviceStatus** (Статуси пристроїв)

Сутність, яка зберігає інформацію про пристрої, зокрема:

- Унікальний ідентифікатор пристрою (**DeviceID**).
- Посилання на користувача (**UserID**).
- Час останньої синхронізації (**LastSync**) і статус активності пристрою (**IsActive**).

3. **Notifications** (Сповіщення)

Сутність для зберігання сповіщень користувача:

- Унікальний ідентифікатор сповіщення (**NotificationID**).
- Заголовок (**Title**) і текст сповіщення (**Message**).
- Статус прочитання (**IsRead**) і час створення (**CreatedAt**).

4. **Tasks** (Завдання)

Сутність для зберігання завдань користувачів:

- Унікальний ідентифікатор завдання (**TaskID**).
- Назва завдання (**Title**) і його опис (**Description**).
- Дата завершення (**DueDate**), статус виконання (**IsCompleted**) і часові мітки (**CreatedAt**, **UpdatedAt**).

5. **TaskLogs** (Журнали завдань)

Сутність для реєстрації дій із завданнями:

- Унікальний ідентифікатор журналу (**LogID**).
- Ідентифікатор завдання (**TaskID**), тип дії (**Action**) і час виконання дії (**Timestamp**).

Основні функції системи:

1. Моніторинг завдань і пристроїв:

- Дані про активність пристроїв та завдання користувачів синхронізуються в реальному часі.
- Кожна подія зберігається в таблицях **TaskLogs** та **DeviceStatus** для аналізу.

2. Управління завданнями:

- Завдання зберігаються в таблиці **Tasks**. Користувачі можуть додавати, редагувати або видаляти завдання через інтерфейс.

3. Обробка сповіщень:

- Система генерує сповіщення про нові завдання, зміну їхнього статусу чи події, пов'язані з пристроями.
- Сповіщення відображаються користувачеві та зберігаються в таблиці **Notifications**.

4. Журнали завдань:

- Таблиця **TaskLogs** зберігає всі дії з завданнями, такі як створення, редагування, виконання чи видалення.

5. Управління статусом пристроїв:

- Дані про пристрої зберігаються в **DeviceStatus**. Відстежується їхня активність і час останньої синхронізації.

Масштабованість та безпека:

- **Масштабованість:** Система підтримує зв'язки "один до багатьох" і "один до одного", що забезпечує її розширення.
- **Безпека:** Дані користувачів і пристроїв захищені сучасними методами шифрування. Політики доступу забезпечують відповідність вимогам GDPR.

Ця структура бази даних забезпечує надійність, зручність у використанні та легкість інтеграції із зовнішніми платформами.

Специфікація API для TaskLamp

Базовий URL:

- Локально: `http://localhost:<PORT>`
- Через тунель: `<NGROK_URL>`

Ендпоінти API

1. Управління користувачами

- **GET /users/:id**

Опис: Отримання інформації про користувача за ідентифікатором.

Параметри:

- id — Унікальний ідентифікатор користувача.

Відповідь:

- Інформація про користувача (username, email).

- **POST /users**

Опис: Створення нового користувача.

Тіло запиту:

```
{
  "username": "string",
  "email": "string",
  "password": "string"
}
```

Відповідь:

- Дані створеного користувача.

- **PUT /users/:id**

Опис: Оновлення даних користувача.

Параметри:

- id — Ідентифікатор користувача.

Тіло запиту:

```
{
  "username": "string",
  "email": "string",
  "password": "string"
}
```

Відповідь:

- Оновлені дані користувача.

2. Управління завданнями

- **GET /tasks**

Опис: Отримання списку всіх завдань.

Відповідь:

- Список завдань.

- **POST /tasks**

Опис: Створення нового завдання.

Тіло запиту:

```
{
  "userId": "int",
  "title": "string",
  "description": "string",
  "dueDate": "datetime",
  "isCompleted": "bool"
}
```

Відповідь:

- Дані створеного завдання.

- **GET /tasks/user/:userId**

Опис: Отримання завдань конкретного користувача.

Параметри:

- userId — Ідентифікатор користувача.

Відповідь:

- Список завдань користувача.

3. Управління налаштуваннями лампи

- **GET /lampsettings**

Опис: Отримання всіх налаштувань лампи.

Відповідь:

- Налаштування лампи.

4. Управління статусами пристроїв

- **GET /devicestatus**

Опис: Отримання статусів усіх пристроїв.

Відповідь:

- Список статусів пристроїв.

5. Управління сповіщеннями

- **GET /notifications**

Опис: Отримання всіх сповіщень.

Відповідь:

- Список сповіщень.

- **POST /notifications**

Опис: Створення нового сповіщення.

Тіло запиту:

```
{
  "userId": "int",
  "title": "string",
  "message": "string",
  "isRead": "bool"
}
```

Відповідь:

- Дані створеного сповіщення.

6. Генерація звітів

- **GET /reports/user/:userId**

Опис: Отримання загальної статистики завдань користувача.

Параметри:

- `userId` — Ідентифікатор користувача.

Відповідь:

- Загальна статистика завдань.

- **GET /reports/user/:userId/overdue**

Опис: Отримання списку прострочених завдань.

Параметри:

- `userId` — Ідентифікатор користувача.

Відповідь:

- Список прострочених завдань.

Ця специфікація відображає поточні можливості **TaskLamp API**, базуючись на завантажених файлах та коді.

ДОДАТОК А

Код програми

Код server.js

```
require('dotenv').config();
const express = require('express');
const { sequelize } = require('./models');

const app = express();
app.use(express.json());

const PORT = process.env.PORT || 3000;

// Імпорт маршрутів
const usersRouter = require('./routes/users');
const tasksRouter = require('./routes/tasks');
const notificationsRouter = require('./routes/notifications');
const deviceStatusRouter = require('./routes/devicestatus');
const lampSettingsRouter = require('./routes/lampsettings');
const reportRouter = require('./routes/lampsettings');

// Реєстрація маршрутів
app.use('/users', usersRouter);
app.use('/tasks', tasksRouter);
app.use('/notifications', notificationsRouter);
app.use('/devicestatus', deviceStatusRouter);
app.use('/lampsettings', lampSettingsRouter);
app.use('/reports', reportRouter);

// Запуск сервера
app.listen(PORT, async () => {
  console.log(`Сервер працює на порту ${PORT}`);
  try {
    await sequelize.authenticate();
    console.log('Підключено до бази даних');
  } catch (error) {
    console.error('Не вдалося підключитися до бази даних:', error);
  }
});
```

Код routes/tasks.js

```
const express = require('express');
const router = express.Router();
const { Task } = require('../models');

// Отримати всі завдання
router.get('/', async (req, res) => {
  try {
    const tasks = await Task.findAll();
    res.json(tasks);
  }
});
```

```

    } catch (error) {
      res.status(500).json({ error: error.message });
    }
  });

  // Отримати завдання за ID
  router.get('/:id', async (req, res) => {
    try {
      const task = await Task.findByPk(req.params.id);
      if (!task) {
        return res.status(404).json({ message: 'Task not found' });
      }
      res.json(task);
    } catch (error) {
      res.status(500).json({ error: error.message });
    }
  });

  // Отримати всі завдання за UserID
  router.get('/user/:userId', async (req, res) => {
    try {
      const tasks = await Task.findAll({
        where: {
          userId: req.params.userId,
        },
      });
      if (tasks.length === 0) {
        return res.status(404).json({ message: 'No tasks found for this user' });
      }
      res.json(tasks);
    } catch (error) {
      res.status(500).json({ error: error.message });
    }
  });

  // Створити нове завдання
  router.post('/', async (req, res) => {
    try {
      const { userId, title, description, dueDate, isCompleted } = req.body;
      const newTask = await Task.create({ userId, title, description, dueDate, isCompleted });
      res.status(201).json(newTask);
    } catch (error) {
      res.status(500).json({ error: error.message });
    }
  });

  // Оновити інформацію про завдання
  router.put('/:id', async (req, res) => {
    try {
      const task = await Task.findByPk(req.params.id);
      if (!task) {
        return res.status(404).json({ message: 'Task not found' });
      }
    }
  });

```

```

    const { title, description, dueDate, isCompleted } = req.body;
    task.title = title || task.title;
    task.description = description || task.description;
    task.dueDate = dueDate || task.dueDate;
    task.isCompleted = isCompleted !== undefined ? isCompleted : task.isCompleted;
    await task.save();
    res.json(task);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

```

// Видалити завдання

```

router.delete('/:id', async (req, res) => {
  try {
    const task = await Task.findByPk(req.params.id);
    if (!task) {
      return res.status(404).json({ message: 'Task not found' });
    }
    await task.destroy();
    res.json({ message: 'Task deleted successfully' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

```

// Пошук завдань за назвою

```

router.get('/search/:title', async (req, res) => {
  try {
    const tasks = await Task.findAll({
      where: {
        title: req.params.title,
      },
    });
    if (tasks.length === 0) {
      return res.status(404).json({ message: 'No tasks found' });
    }
    res.json(tasks);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

```

// Пошук завдань за статусом виконання

```

router.get('/status/:isCompleted', async (req, res) => {
  try {
    const isCompleted = req.params.isCompleted === 'true';
    const tasks = await Task.findAll({
      where: {
        isCompleted,
      },
    });
    res.json(tasks);
  } catch (error) {

```

```
    res.status(500).json({ error: error.message });
  }
});

module.exports = router;
```